.NET Skill Architecture

This article provides an overview of some of the basic building blocks of a .NET skill. It describes the required elements of your C# skill file, and it provides examples of some of the different ways to perform common tasks in your .NET skill code, such as issuing commands to Misty, getting data from the robot's sensors and events, and handling that data in your code.

Skill Class Members

`IMistySkill`

Each .NET skill must implement the `IMistySkill` interface. This interface loads the connection between Misty and the background task that runs the skill code, sets the name of the skill as it appears in the Skill Runner web page, and defines what happens when the skill starts and stops.

```
namespace MySkill
{
    public class MySkill : IMistySkill
    {
        // Insert skill code here
    }
}
```

C#

`IRobotMessenger`

Within the skill class, you must also instantiate a variable to hold the `IRobotMessenger` interface. We use this interface to issue commands to the robot from our .NET skill code.

```
// Instantiate a local variable to hold the Misty Robot interface. In
// this case we use _misty, but you can call it whatever you like.
private IRobotMessenger _misty
```

C#

`NativeRobotSkill`

Each .NET skill must also implement a new `NativeRobotSkill`. The constructor for this class requires two parameters:

- a name for the skill (used in the Skill Runner web page)
- a unique GUID (used by Misty to identify your skill)

In this example, we pass in a name of `MySkill` and set a unique GUID.

```
public NativeRobotSkill Skill { get; private set; } = new
NativeRobotSkill("MySkill", "07b7b656-8d3e-4b94-9fa1-c9a010cfdc9b");
```

C#

The complete list of required and optional properties for the `NativeRobotSkill` class includes:

- `AllowedCleanupTimeInMs` (uint) - Optional. The amount of time given to perform cleanup tasks when a skill is cancelled or times out. Depending on this value, there may be some delay before you can restart a skill after it cancels or times out. Defaults to 2000 milliseconds. Maximum value is 10000 milliseconds.

- `UniqueId` (Guid) - Required. The unique GUID that Misty uses to identify your skill.
- `Name` (string) - Required. The name of the skill. Appears in the Skill Runner web page.
- `Description` (string) - Optional. A description of your skill.
- `BroadcastMode` (BroadcastMode) - Optional. Configures the contents and frequency of messages Misty sends to `SkillData` WebSocket listeners.
    - `Off` - The skill does not send `SkillData` messages. This is the default setting.
    - `Debug` - The skill sends error and debug messages to `SkillData` listeners.
    - `Verbose` - In addition to error and debug messages, the skill sends a message to `SkillData` listeners for each command that Misty receives.
- `StartupRules` (`IList<NativeStartupRule>`) - Optional. Configures how a skill can start. The following rules are currently implemented:
    - `Manual` - Allows a user to start the skill from the Skill Runner web page or by sending a request to the `RunSkill` endpoint in Misty's REST API. By default, all skills are implemented with the Manual startup rule.
    - `Startup` - Configures the skill to start as soon as Misty boots up. **Note:** Misty runs **all** skills set to run on startup when she boots up.
- `TimeoutInSeconds` (int) - Optional. The number of seconds a skill runs before it times out. Defaults to 600 seconds (or 10 minutes).
- `SkillStorageLifetime` (`SkillStorageLifetime`) - Determines how long the system saves the shared data this skill creates.
    - `Skill` - The data clears when the skill stops running.
    - `Reboot` - The data clears the next time Misty reboots (default).
    - `LongTerm` - The data persists across reboots and remains available until removed from the robot with the `DeleteSharedData` (.NET) or `misty.Remove()` (JavaScript) command.
- `ReadPermissions` (`IList<string>`) - A list of `UniqueId`s for each skill that is allowed to read the shared data this skill creates. If empty, any skill can read this skill's shared data. If an empty Guid, no outside skills can read this skill's shared data. If the list includes one or more `UniqueId`s, only those skills (and this one) can read this skill's shared data.
- `WritePermissions` (`IList<string>`) - A list of `UniqueId`s for each skill that is allowed to create, update, and remove shared data associated with this skill. If empty, any skill can write to this skill's shared data. If an empty Guid, no outside skills can write to this skill's shared data. If the list includes one or more `UniqueId`s, only those skills (and this one) can write to this skill's shared data.
- `StartPermissions` (`IList<string>`) - A list of `UniqueId`s for each skill that is allowed to start or cancel this skill. If empty, any skill can start or cancel this skill. If an empty Guid, no skill can start or cancel this skill. If the list includes one or more `UniqueId`s, only those skills (and this one) can start or stop this skill.
- `TriggerPermissions` (`IList<string>`) - A list of `UniqueId`s for each skill that is allowed to trigger user events in this skill. If empty, all skills can trigger events in this skill. If an empty Guid, only this skill can trigger user events within itself. If the array contains one or more `UniqueId`s, only those skills (and this skill) can trigger user events within this skill.

LoadRobotConnection

Each .NET skill must also call the `LoadRobotConnection()` method. This method configures the robot messenger interface when the skill starts. For example:

```
public void LoadRobotConnection(IRobotMessenger robotInterface)
{
    _misty = robotInterface;
}
```

C#

OnStart

The `OnStart()` method includes the code that runs when your background task receives a message from the robot or a user to start the skill (for example, when you send a request to start the skill from the Skill Runner web page). In many cases, the majority of your skill code exists inside the body of the `OnStart()` method.

```
public void OnStart(object sender, IDictionary<string, object>
parameters)
{
    // Your skill code goes here!
    _misty.ChangeLED(0, 255, 0) // Green means start!
}
```

C#

> **Tip:** The `OnStart()` method accepts an optional `parameters` argument. This argument populates your skill with the optional payload of key/value pairs that you can send to Misty when you issue a command to start a skill.

OnCancel

The `OnCancel()` method runs when your skill receives a cancellation message (for example, when you send a request to stop the skill from the Skill Runner web page). Use this method to stop any ongoing processes and return the robot to a default state before the skill shuts down. By default, each .NET skill has 2 seconds to clean up any running processes. You can increase or decrease this time time by passing in a custom value for the `AllowedCleanupTimeInMs` property when you implement the `NativeRobotSkill` class.

> **Note:** Any registered events are automatically unregistered when a skill cancels.

```
public void OnCancel(object sender, IDictionary<string, object>
parameters)
{
    _misty?.SkillLogger.LogInfo($"MySkill : OnCancel called");
}
```

C#

OnTimeout

The `OnTimeout()` method runs when the skill times out. As with `OnCancel()`, you can use the `OnTimeout()` method to return the robot to a default state.

```
public void OnTimeout(object sender, IDictionary<string, object>
parameters)
{
    _misty?.SkillLogger.LogInfo($"MySkill : OnTimeout called");
}
```

C#

OnPause

The `OnPause()` method runs when the skill is paused. You can use this method to save any states and other items such that a call to the `OnResume()` method starts the skill where it left off. After the actions in this method are complete, the robot shuts down the task. If you choose not to use this functionality, you can use the `OnPause()` method to cancel the skill.

```
public void OnPause(object sender, IDictionary<string, object>
parameters)
```

```
{
    // In this example, Pause is not implemented by default and
    // OnPause simply calls OnCancel
    _misty.SkillLogger.LogVerbose($"MySkill : OnPause called");
    OnCancel(sender, parameters);
}
```

C#

OnResume

The `OnResume()` method is called when the skill is resumed. This method loads up any states and other items saved when the skill is paused and starts up the task. If you choose not to use this functionality, you can use the `OnResume()` method to start the skill.

```
public void OnResume(object sender, IDictionary<string, object>
parameters)
{
    // In this example, Resume is not implemented by default and it
    // OnResume simply calls OnStart
    _misty.SkillLogger.LogVerbose($"MySkill : OnResume called");
    OnStart(sender, parameters);
}
```

C#

Command Syntax

There are two main techniques for issuing commands in your skill code. Both techniques require calling commands as methods on the variable that holds Misty's interface.

The first technique is to issue the command to Misty and continue execution without waiting for response data from the robot. When you use this technique, you have the option to specify a callback function that runs when the command completes.

```
// Changes Misty's LED to green; calls CommandCallback when complete
_misty.ChangeLED(0, 255, 0, CommandCallback);
```

C#

Alternately, you can call an asynchronous version of the command that waits for the response before continuing execution:

```
// Changes LED to green and waits for response from the robot.
var result = await _misty.ChangeLEDAsync(0, 255, 0);
```

C#

This technique can be useful when you need to retrieve data from Misty, such as a list of her image files:

```
// Gets image list for use in the skill
_imageList = (await _misty.GetImageListAsync())?.Data;
```

C#

> **Tip:** Until comprehensive documentation on the structure of each class in Misty's .NET SDK libraries is available, use IntelliSense to see the list of available commands and their parameters in Visual Studio.

Repeating Actions

As you build skills for Misty, you may wish to program her to repeat a set of actions. There are a few patterns you can use to accomplish this. The following examples show two different techniques for coding Misty to change the color of her chest LED on a loop. The first example uses a `while` loop, and the second uses a timer to trigger a callback to invoke the relevant command. Note that these examples assume the robot interface is assigned to a local variable called `_misty`.

Using while and Wait

This example shows how to use `while` to invoke the `ChangeLED` command on a loop. On each iteration, we call the `Wait()` method to pause execution for 100 milliseconds. Unless the skill is cancelled while in a paused state, the `Wait()` method returns a value of `true`. This means our loop repeats until the skill times out or is cancelled. In the body of the loop, we call the `ChangeLED()` command, passing in randomized values for the RGB arguments on each iteration. In your .NET skill code, you would place this block in the body of the `OnStart()` method.

```
// Instantiates a new random number generator
Random randomGenerator = new Random();

// Pause for 100 ms. If cancellation token is set during this time,
// exit Pause and the method. If not, invoke change LED command.
while (_misty.Wait(100))
{
    // Sends Misty a command to change her LED
    _misty.ChangeLED((uint)randomGenerator.Next(0, 256),
(uint)randomGenerator.Next(0, 256), (uint)randomGenerator.Next(0, 256),
null);
}
```

C#

Using a Timer

You can also repeat actions by implementing a `Timer` and triggering a callback function after a set duration. In this example, we create a `Timer` called `_heartbeatTimer` as a member of our skill class:

```
private Timer _heartbeatTimer;
```

C#

Then, in the body of the `OnStart()` method, we instantiate the timer. In this example, the timer starts after 1000 milliseconds, then triggers the `HeartBeat` callback every 250 seconds.

```
// Instantiate our new Timer as _heartbeatTimer. This example waits one
// second before starting, then invokes the callback every 250 ms until
// skill times out or is cancelled
_heartbeatTimer = new Timer(HeartbeatCallback, null, 1000, 250);
```

C#

We define the `HeartBeatCallback()` method as a helper function elsewhere in our skill code. When called, this method issues a `Wait()` command to see if the skill is cancelled, and then issues a `ChangeLED()` command with randomized RGB values.

```
/// Callback method for the heartbeat timer event
private void HeartbeatCallback(object data)
{
    // Calls on Wait and returns if skill is cancelled
    if(!_misty.Wait(0)) { return; }
    // Issues a ChangeLED command
    _misty.ChangeLED((uint)_randomGenerator.Next(0, 256),
(uint)_randomGenerator.Next(0, 256), (uint)_randomGenerator.Next(0,
```

```
256), null);
    // Prints a message to the skill's log file
    _misty.SkillLogger.LogVerbose($"Change LED called after heartbeat
callback");
}
```

C#

Data Handling

Most of the skills you write for Misty require you to determine how Misty should react to information she gathers from her sensors and from third-party APIs. The topics in this section describe techniques you can use to can access, filter, and validate sensor and event data in your skill code, using examples from the `InteractiveMistySkill` and `ForceDriving` sample skills.

Registering & Unregistering Events

You can create a listener for event messages by issuing a command to register for a specific event type. In most cases, commands for registering event listeners can take the following arguments:

- `eventCallback` - The callback the skill triggers when it receives a message from the associated event type.
- `debounce` - How long (in milliseconds) to pause between event messages. Setting this to a value other than `0` can cause you to miss messages.
- `keepAlive` - If `false`, the skill unregisters the event after the first event message is received, thus triggering the callback function only once.
- `validations` - An optional list of validations to apply to your event listener. Validations allow you to filter out unwanted event messages. Read more about validations below.
- `eventName` - An optional name for this event listener. Giving an event listener a specific name allows you to unregister that specific event listener elsewhere in your skill.
- `commandCallback` - An optional callback function to invoke when the robot sends a message to your skill acknowledging it has received the command. This acknowledgement message is sent separately from event message data.

The following example from the `InteractiveMistySkill` shows the command we use to register for messages from Misty's capacitive touch sensors. When the skill gets a capacitive touch event message, it passes that data into the `CapTouchCallback()` method. We set the `debounce` to be `true`, so that the skill continues to listen for messages until we unregister this event.

```
// Registers for event messages from Misty's capacitive touch sensors.
_misty.RegisterCapTouchEvent(CapTouchCallback, 0, true, null, null,
null);
```

C#

When a cap touch event is triggered, the event message is passed into the `CapTouchCallback()`, shown here:

```
private void CapTouchCallback(ICapTouchEvent capTouchEvent)
{
    if(capTouchEvent.IsContacted)
    {
        switch (capTouchEvent.SensorPosition)
        {
            case CapTouchPosition.Back:
                _misty.PlayAudio("s_Love.wav", 100, null);
                _misty.DisplayImage("e_Love.jpg", 1 , null);
                break;
            case CapTouchPosition.Front:
                _misty.PlayAudio("s_Amazement.wav", 100, null);
                _misty.DisplayImage("e_Amazement.jpg", 1, null);
```

```
                    break;
                case CapTouchPosition.Right:
                    _misty.PlayAudio("Joy.wav", 100, null);
                    _misty.DisplayImage("JoyGoofy3.jpg", 1, null);
                    break;
                case CapTouchPosition.Left:
                    _misty.PlayAudio("e_Terror.jpg", 100, null);
                    _misty.DisplayImage("s_Fear.wav", 1, null);
                    break;
                case CapTouchPosition.Scruff:
                    _misty.PlayAudio("s_Rage.wav", 100, null);
                    _misty.DisplayImage("e_Rage4.jpg", 1, null);
                    break;
                case CapTouchPosition.Chin:
                    _misty.PlayAudio("s_Sleepy.wav", 100, null);
                    _misty.DisplayImage("e_Sleepy2.jpg", 1, null);
                    break;
            }
        }
    }
```

C#

When you want to unregister all of the event listeners in a skill, you can call the `UnregisterAllEvents()` command as follows:

```
// Unregisters all events
_misty.UnregisterAllEvents(UnregisterCallback);
```

C#

Unregistering events also triggers a callback you define in your skill code (in this case, `UnregisterCallback()`):

```
private void UnregisterCallback(IRobotCommandResponse response)
{
      // Writes a message to the skill's log file.
     _misty.SkillLogger.Log($"InteractiveMistySkill : UnregisterCallback
called");
}
```

C#

In addition to registering generalized event listeners, you can also assign event listeners unique names. This allows you to create multiple listeners for the same type of event message, and it gives you the option to unregister specific event listeners while leaving others active. Using this technique to register a bump sensor event called `MyBumpSensorName` would look like this:

```
//Create an event with a specific name so we can unregister it when
// needed using that name
_misty.RegisterBumpSensorEvent(BumpCallback, 0, true, null,
"MyBumpSensorName", null);
```

C#

Unregistering the `MyBumpSensorName` event listener would look like this. (Note that this also triggers our `UnregisterCallback()` function.)

```
//Unregister the bump sensor
_misty.UnregisterEvent("MyBumpSensorName", UnregisterCallback);
```

C#

As an alternative to the above techniques, you can register events without specifying a callback function. When you do this, you must use the following pattern:

```
// Registers for bump event without a callback
_misty.RegisterBumpSensorEvent(0, true, null, null, null);
// Attach ProcessBumpEvent method to BumpSensorEventReceived events.
_misty.BumpSensorEventReceived += ProcessBumpEvent;
```

C#

When you use this technique, the event handler (for bump sensor events, `ProcessBumpEvent`) is attached to the `BumpSensorEventReceived` event. In the `InteractiveMistySkill` sample, the `ProcessBumpEvent` is defined as follows:

```
private void ProcessBumpEvent(object sender, IBumpSensorEvent bumpEvent)
{
    if (!bumpEvent.IsContacted)
    {
        _misty.Stop(null);
        switch (bumpEvent.SensorPosition)
        {
            case BumpSensorPosition.FrontRight:
                _misty.PlayAudio("s_PhraseHello.wav", 100, null);
                break;
            case BumpSensorPosition.FrontLeft:
                _misty.PlayAudio("s_PhraseUhOh.wav", 100, null);
                break;
            case BumpSensorPosition.BackRight:
                _misty.PlayAudio("s_Love.wav", 100, null);
                break;
            case BumpSensorPosition.BackLeft:
                _misty.PlayAudio("s_Boredom.wav", 100, null);
                break;
        }
    }
}
```

C#

Validating Event Data

In some cases you may find it useful to validate data from event messages before passing that data into a callback function or event handler. This is useful when you want Misty to respond to an event only when it meets specific criteria – for example, if you want Misty to play a sound when you press her front right bump sensor, but not to react when you release that sensor, and to remain silent when you interact with any of her other bump sensors.

To allow for this, several of Misty's event registration commands accept an optional `validations` parameter. You can use this parameter by assigning the validations you want to check against to a validation list specific to the corresponding event type. Then, when you register for the event, you pass in this validation list for the `validations` argument.

The following example from the `ForceDriving` sample skill shows how to use validations when registering for time-of-flight messages from Misty's front right time-of-flight sensor. It applies two validations to the event listener:

- the first validates that the value of `DistanceInMeters` is less than or equal to `0.3`
- the second validates that the `SensorName` string is equal to `FrontRight`

```
List<TimeOfFlightValidation> tofFrontRightValidations = new
List<TimeOfFlightValidation>();
tofFrontRightValidations.Add(new TimeOfFlightValidation { Name =
TimeOfFlightFilter.DistanceInMeters, Comparison =
ComparisonOperator.LessThanOrEqual, ComparisonValue = 0.3 });
tofFrontRightValidations.Add(new TimeOfFlightValidation { Name =
TimeOfFlightFilter.SensorName, Comparison =
ComparisonOperator.Equal, ComparisonValue =
TimeOfFlightPosition.FrontRight });
```

```
_misty.RegisterTimeOfFlightEvent(TOFRangeCallback, 300, true,
tofFrontRightValidations, null, null);
```

C#

As a result, the registered event listener only triggers the `TOFRangeCallback()` method when the validation conditions are met.

Skill Logging

Misty's .NET SDK provides tools for writing messages to log files associated with a specific .NET skill. Each time you run a .NET skill, Misty creates a `.log` file with the name of the skill and a timestamp. Skill log files are stored separately from the general purpose log file, which many of the robot's systems can write to.

You can write your own messages to the log file for a skill by using the `SkillLogger.Log()` method. For example:

```
// Writes a message to the skill's log file
_misty.SkillLogger.Log($"This message appears in the log file.");
```

C#

Accessing Skill Log Files

Misty stores log files for .NET skills on her local file system. You can access this file system by connecting to Misty's file server over your local network connection. Once connected, look for skill log files in the following directory:

```
c$/Data/Misty/SDK/Logs
```

File System Access

It is possible to use files stored on Misty's file system from within a .NET skill. Accessing files in the file system can be done in different ways, depending on the location of the file. In particular, files stored in Windows *known folders* must be accessed in specific ways, while files in user-created locations do not have such restrictions.

Accessing Files in Known Folders

The Window's file system uses *known folders* with specific access restrictions for security reasons. These known folders include:

- Documents
- Music
- Pictures
- Videos

When using files in these known folders in your .NET skill, you must access them using the `Windows.Storage` interface. Using the `System.IO` interface results in permission denied restrictions.

The following example shows how to access a `.json` file in the Documents known folder in a project created with the skill template. It assumes you created your project using the C# skill template.

Start by including a `documentsLibrary` line in the `Capabilities` section of the `Package.appxmanifest` file for your project:

```
<Capabilities>
      <Capability Name="internetClient" />
```

```
      <uap:Capability Name="documentsLibrary" />
      <rescap:Capability Name="broadFileSystemAccess" />
</Capabilities>
```

XML

Then, add a new `Extension` to the `Extensions` section of the `Package.appxmanifest` file to identify the file extension you wish to use. In this example, we use `.json`.

```
<uap:Extension Category="windows.fileTypeAssociation" Executable="Dummy.exe"
EntryPoint="Dummy.app">
      <uap:FileTypeAssociation Name="filetypes">
      <uap:SupportedFileTypes>
            <uap:FileType>.json</uap:FileType>
      </uap:SupportedFileTypes>
      </uap:FileTypeAssociation>
</uap:Extension>
```

XML

Include the following reference in your skill code file:

```
using Windows.Storage;
```

C#

You must use the `Windows.Storage` methods to access the file in your skill code. For example:

```
var storageLibrary = await
StorageLibrary.GetLibraryAsync(KnownLibraryId.Documents);
var storageFile = await
storageLibrary.SaveFolder.GetFileAsync(filename);
string fileData = await FileIO.ReadTextAsync(storageFile);
```

C#

Accessing Files in Arbitrary Locations

You can access files in arbitrary location (such as a user-defined folder of `c:\temp`) using either the `Windows.Storage` interface or the general `System.IO` interface. For example:

```
if (!File.Exists("c:\\temp\\testfile.txt"))
{
    File.WriteAllLines("testing content");
}
```

C#

Running a .NET Skill

Once you've installed a .NET skill on Misty, you can start and stop your skill by using the Skill Runner interface, or by using a REST client like Postman to issue `RunSkill` and `CancelSkill` commands. Alternately, you can write your own code to issue these commands from an external device.

To start a .NET skill, issue a `RunSkill` POST command with the following syntax. The value of the `Skill` key must be the GUID set in your `NativeRobotSkill` constructor. For example:

```
Endpoint: POST http://<robot-ip-address>/api/skills/start
{
    "Skill": "<Skill GUID>"
}
```

To stop a skill, send the `CancelSkill` POST command to the following endpoint, passing in the GUID for the value of the `Skill` parameter in the payload.

```
Endpoint: POST http://<robot-ip-address>/api/skills/cancel
{
    "Skill": "<Skill GUID>"
}
```

To stop all running skills, send the same POST command with an empty payload (no skill name specified):

```
Endpoint: POST http://<robot-ip-address>/api/skills/cancel
```